



# 第19课时：结构体

自定义数据类型 · 信息封装 · 数据组织

结构体的概念、定义与成员访问、结构体数组、结构体作为函数参数



# 上节课回顾



上节课我们学习了**递归入门**：

- **递归概念**：函数调用自身，把大问题拆成同类型的更小问题
- **两要素**：**边界条件**（何时停止）+ **递归公式**（如何缩小）
- **经典示例**：阶乘  $n! = n * (n-1)!$ 、斐波那契  $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$
- **综合例题**：递归求和、数字之和、最大公约数、汉诺塔

本节课我们学习**结构体**——C++ 中自定义数据类型的核心工具。

# 为什么需要结构体？



**问题：**一个班有 30 个学生，每人有姓名、年龄、成绩三项数据，怎么存储？

## 方案一：用多个数组

```
string names[30];  
int ages[30];  
double scores[30];  
// 第 i 个学生的信息分散在三个数组里
```

⚠ 同一个学生的信息散落在不同数组中，**数据不内聚**，容易错乱。

## 方案二：用一个变量描述一个学生的全部信息

```
// 如果能把 name、age、score "打包" 在一起就好了.....
```

💡 这正是**结构体**的作用——把多个相关数据"打包"成一个新的类型。

# 结构体的概念



**结构体 (struct)** 是一种**自定义数据类型**, 可以把多个不同类型的变量组合在一起。

**类比:**

- `int` 是一个整数——C++ 内置类型
- `string` 是一串字符——C++ 标准库类型
- `Student` 是一个学生 (含姓名、年龄、成绩) ——**你自己定义的类型**

**生活中的结构体:**

- 一张身份证 = 姓名 + 性别 + 民族 + 出生日期 + 地址 + 身份证号
- 一本图书 = 书名 + 作者 + 出版社 + 价格 + ISBN
- 一个游戏角色 = 名字 + 等级 + 生命值 + 攻击力 + 防御力

# 结构体的定义



使用 `struct` 关键字定义结构体类型：

```
struct Student {  
    string name;    // 姓名  
    int age;        // 年龄  
    double score;   // 成绩  
}; // ← 注意结尾的分号!
```

## 语法要点：

- `struct` + 类型名（如 `Student`）+ `{ ... }`
- 花括号内是**成员变量**（也叫"字段"）
- 每个成员像普通变量一样声明
- **结尾分号不能忘！**

💡 定义了 `Student` 类型后，就可以像 `int`、`string` 一样使用它。

# 声明结构体变量



定义类型后，声明变量：

```
// 先定义结构体类型
struct Student {
    string name;
    int age;
    double score;
};

int main() {
    Student s1;           // 声明一个 Student 类型的变量
    Student s2;           // 再声明一个
    Student class[30];     // 声明一个结构体数组
    return 0;
}
```

💡 `Student` 是**类型名**，`s1` 是**变量名**——就像 `int` 是类型，`x` 是变量。

# 成员访问——点运算符



用**点运算符** `.` 访问结构体变量的成员：

```
Student s1;  
s1.name = "张三";  
s1.age = 15;  
s1.score = 95.5;  
  
cout << s1.name << " " << s1.age << "岁 "  
      << s1.score << "分" << endl;  
// 输出：张三 15岁 95.5分
```

**规则：** `变量名.成员名` 就像一个普通变量，可以赋值、读取、参与运算。



# 结构体的初始化



## 方式一：声明后逐个赋值

```
Student s1;  
s1.name = "张三";  
s1.age = 15;  
s1.score = 95.5;
```

## 方式二：花括号列表初始化（推荐）

```
Student s2 = {"李四", 16, 88.0};  
// 按成员定义顺序依次赋值
```

## 方式三：声明时直接初始化

```
Student s3{"王五", 14, 100.0};
```

⚠ 花括号初始化时，值的顺序必须与成员定义顺序**一一对应**。

# 结构体与输入输出



通过 `cin` / `cout` 逐个成员输入输出：

```
#include <bits/stdc++.h>
using namespace std;

struct Student {
    string name;
    int age;
    double score;
};

int main() {
    Student s;
    cin >> s.name >> s.age >> s.score;
    cout << "姓名: " << s.name << endl;
    cout << "年龄: " << s.age << endl;
    cout << "成绩: " << s.score << endl;
    return 0;
}
// 输入: 张三 15 95.5
// 输出:
// 姓名: 张三
// 年龄: 15
// 成绩: 95.5
```

# 结构体数组



结构体可以组成数组，适合批量管理数据：

```
#include <bits/stdc++.h>
using namespace std;

struct Student {
    string name;
    int age;
    double score;
};

int main() {
    Student cls[3] = {
        {"张三", 15, 95.5},
        {"李四", 16, 88.0},
        {"王五", 14, 100.0}
    };
    // 访问第 2 个学生的成绩
    cout << cls[1].name << ": " << cls[1].score << endl;
    // 输出：李四：88
    return 0;
}
```



`cls[i].name` 表示数组中第 `i` 个元素的 `name` 成员。

# 结构体数组的遍历



用循环遍历结构体数组：

```
#include <bits/stdc++.h>
using namespace std;

struct Student {
    string name;
    int age;
    double score;
};

int main() {
    int n;
    cin >> n;
    Student cls[100];
    for (int i = 0; i < n; i++) {
        cin >> cls[i].name >> cls[i].age >> cls[i].score;
    }
    // 输出所有学生信息
    for (int i = 0; i < n; i++) {
        cout << cls[i].name << " " << cls[i].age
              << " " << cls[i].score << endl;
    }
    return 0;
}
```

# 结构体作为函数参数——值传递



结构体可以作为函数参数传递，默认是**值传递**（传副本）：

```
void printStudent(Student s) {    // 传值，s 是副本
    cout << s.name << " " << s.age << " " << s.score << endl;
}

void tryModify(Student s) {      // 修改副本，不影响原变量
    s.score = 0;
}

int main() {
    Student s1 = {"张三", 15, 95.5};
    printStudent(s1);    // 输出: 张三 15 95.5
    tryModify(s1);
    printStudent(s1);    // 输出: 张三 15 95.5 ← 没变!
    return 0;
}
```

⚠ 和普通变量一样，值传递中修改形参**不影响**实参。

# 结构体作为函数参数——引用传递



若需要在函数内修改结构体，使用**引用传递** `&`：

```
void birthday(Student &s) {           // 引用传递，直接操作原变量
    s.age++;
}

int main() {
    Student s1 = {"张三", 15, 95.5};
    birthday(s1);
    cout << s1.name << " 现在年龄: " << s1.age << endl;
    // 输出: 张三 现在年龄: 16 ← 改了!
    return 0;
}
```

💡 加 `&` 就是引用传递，函数内修改**直接影响**原变量。

# 结构体作为函数返回值



函数也可以返回结构体：

```
Student createStudent(string name, int age, double score) {  
    Student s;  
    s.name = name;  
    s.age = age;  
    s.score = score;  
    return s;  
}  
  
int main() {  
    Student s1 = createStudent("赵六", 15, 90.0);  
    cout << s1.name << " " << s1.score << endl;    // 赵六 90  
    return 0;  
}
```

💡 这样可以像"工厂"一样批量创建结构体变量。

# 例题 1：学生信息管理



## 题目描述

输入 `n` 个学生的信息（姓名、年龄、成绩），输出成绩最高的学生信息。

### 输入输出示例：

- 输入：3，然后 张三 15 95.5 / 李四 16 88.0 / 王五 14 100.0
- 输出：王五 14 100

## 解题思路

1. 定义结构体 `Student`，包含 `name`、`age`、`score`
2. 声明结构体数组 `cls[100]`，读入 `n` 个学生信息
3. 用"打擂台"算法找成绩最高者：初始化 `maxIdx = 0`，遍历数组，若 `cls[i].score > cls[maxIdx].score` 则更新 `maxIdx`
4. 输出 `cls[maxIdx]` 的信息



# 例题 1：学生信息管理 —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

struct Student {
    string name;
    int age;
    double score;
};

int main() {
    int n;
    cin >> n;
    Student cls[100];
    for (int i = 0; i < n; i++) {
        cin >> cls[i].name >> cls[i].age >> cls[i].score;
    }
    int maxIdx = 0;
    for (int i = 1; i < n; i++) {
        if (cls[i].score > cls[maxIdx].score) {
            maxIdx = i;
        }
    }
    cout << cls[maxIdx].name << " "
         << cls[maxIdx].age << " "
         << cls[maxIdx].score << endl;
    return 0;
}
```

# 例题 2：成绩统计



## 题目描述

输入 `n` 个学生的姓名和成绩，计算平均成绩（保留 2 位小数），并输出成绩高于平均分的学生姓名。

### 输入输出示例：

- 输入：3，然后 张三 90 / 李四 60 / 王五 80
- 输出：平均分 76.67，高于平均分的学生：张三 王五

## 解题思路

1. 定义结构体 `Student`（只需 `name` 和 `score`）
2. 读入 `n` 个学生，同时累加成绩 `sum += cls[i].score`
3. 计算平均分 `avg = sum / n`
4. 再次遍历数组，若 `cls[i].score > avg`，输出 `cls[i].name`
5. 注意输出格式：用 `fixed` 和 `setprecision(2)` 保留 2 位小数



## 例题 2：成绩统计 —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

struct Student {
    string name;
    double score;
};

int main() {
    int n;
    cin >> n;
    Student cls[100];
    double sum = 0;
    for (int i = 0; i < n; i++) {
        cin >> cls[i].name >> cls[i].score;
        sum += cls[i].score;
    }
    double avg = sum / n;
    cout << fixed << setprecision(2) << avg << endl;
    for (int i = 0; i < n; i++) {
        if (cls[i].score > avg) {
            cout << cls[i].name << " ";
        }
    }
    cout << endl;
    return 0;
}
```

💡 **两趟遍历：**第一趟读入+求和，第二趟筛选。这是数组的经典模式。

# 例题 3：点与距离



## 题目描述

输入两个二维点  $(x_1, y_1)$  和  $(x_2, y_2)$ ，计算它们之间的欧几里得距离（保留 4 位小数）。

### 输入输出示例：

- 输入：0 0 3 4
- 输出：5.0000

## 解题思路

1. 定义结构体 `Point`，包含 `x` 和 `y` 两个 `double` 成员
2. 声明两个 `Point` 变量 `p1`、`p2`，读入坐标
3. 定义函数 `double dist(Point a, Point b)`，计算距离
4. 距离公式： $\text{sqrt}((a.x-b.x)^2 + (a.y-b.y)^2)$
5. 用 `fixed` 和 `setprecision(4)` 保留 4 位小数输出

# 例题 3：点与距离 —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

struct Point {
    double x;
    double y;
};

double dist(Point a, Point b) {
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    return sqrt(dx * dx + dy * dy);
}

int main() {
    Point p1, p2;
    cin >> p1.x >> p1.y >> p2.x >> p2.y;
    cout << fixed << setprecision(4)
         << dist(p1, p2) << endl;
    return 0;
}
// 输入: 0 0 3 4
// 输出: 5.0000
```

💡 结构体作为函数参数，代码更清晰：`dist(p1, p2)` 比 `dist(x1,y1,x2,y2)` 更易读。

# 例题 4：图书管理



## 题目描述

输入 `n` 本图书的信息（书名、作者、价格），按价格从高到低排序后输出。

### 输入输出示例：

- 输入： `3`，然后 `C++入门 张三 59.9` / `算法竞赛 李四 89.0` / `Python 王五 39.9`
- 输出：按价格降序排列的三本书

## 解题思路

1. 定义结构体 `Book`，包含 `title`、`author`、`price`
2. 读入 `n` 本图书到结构体数组
3. 用**选择排序**按 `price` 降序排列：外层从 `i=0` 到 `n-2`，内层找 `i` 到 `n-1` 中价格最高的下标 `maxJ`，交换 `cls[i]` 和 `cls[maxJ]`
4. 遍历输出排序后的结果
5. 注意：结构体变量可以直接用 `=` 赋值（整体复制），无需逐个成员交换



# 例题 4：图书管理 —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

struct Book {
    string title;
    string author;
    double price;
};

int main() {
    int n;
    cin >> n;
    Book books[100];
    for (int i = 0; i < n; i++) {
        cin >> books[i].title >> books[i].author
            >> books[i].price;
    }
    // 选择排序: 按 price 降序
    for (int i = 0; i < n - 1; i++) {
        int maxJ = i;
        for (int j = i + 1; j < n; j++) {
            if (books[j].price > books[maxJ].price) {
                maxJ = j;
            }
        }
        Book temp = books[i];
        books[i] = books[maxJ];
        books[maxJ] = temp;
    }
    for (int i = 0; i < n; i++) {
        cout << books[i].title << " " << books[i].author
            << " " << books[i].price << endl;
    }
    return 0;
}
```

💡 结构体可以直接用 `=` 赋值。编译器会自动复制所有成员——非常方便。

# 常见错误与调试



| 错误类型   | 示例/说明                                                                  | 解决方法                                                  |
|--------|------------------------------------------------------------------------|-------------------------------------------------------|
| 忘记结尾分号 | <code>struct S { ... }</code> ← 少了 <code>;</code>                      | 结构体定义结尾必须有分号 <code>};</code>                          |
| 点与箭头混淆 | 用 <code>-&gt;</code> 访问普通变量成员                                          | 普通变量用 <code>.</code> ，指针用 <code>-&gt;</code> （本课不用指针） |
| 成员名写错  | <code>s.Name</code> 拼写与定义不一致                                           | 成员名区分大小写，与定义完全一致                                      |
| 初始化顺序错 | <code>{"张三", 95.5, 15}</code> （age 和 score 反了）                         | 花括号初始化按定义顺序一一对应                                       |
| 值传递误改  | 函数内改了形参，以为改了原变量                                                        | 需要修改原变量时加 <code>&amp;</code> 引用传递                     |
| 数组越界   | <code>cls[100]</code> 但 <code>n = 100</code> 时访问 <code>cls[100]</code> | 数组大小留余量，如 <code>cls[105]</code>                       |



# 知识小结



- ✓ **结构体概念**：自定义数据类型，把多个相关变量"打包"在一起
- ✓ **定义语法**：`struct 类型名 { 成员列表 };`（结尾分号！）
- ✓ **成员访问**：用 `.` 运算符，`变量名.成员名` 像普通变量一样使用
- ✓ **结构体数组**：`Student cls[100]`，用 `cls[i].name` 访问第 `i` 个元素的成员
- ✓ **函数参数**：值传递传副本（不影响原变量）；引用传递 `&` 可修改原变量
- ✓ **结构体赋值**：可以用 `=` 直接整体复制，编译器自动处理所有成员
- ✓ **实际应用**：学生管理、图书管理、几何计算——把数据组织得更清晰

# 下节课预告



## 下一课时：入门综合练习（第20课时）

- 回顾全学期知识点：从变量到结构体
- 综合题解题思路：**分析问题** → **设计算法** → **编码实现** → **测试调试**
- 阶段性教学检测

**"结构体是 C++ 数据组织的核心工具，掌握它就能处理更复杂的现实问题。"**

# 谢谢大家！

结构体让数据有了"形状"，程序有了"灵魂"！

## Q & A

